Assignment 1

Question 7

Problem Statement:
Write a shell script which displays vendor id, model name, cpu MHz, cache size information about the processor present in your computer. [Hint: most of this information can be obtained by reading the file /proc/cpuinfo.]

p7.sh

```
#!/bin/bash

# check if /proc/cpuinfo is a file or exists
if [ ! -f /proc/cpuinfo ]; then
    exit 1
fi

while true; do
    clear

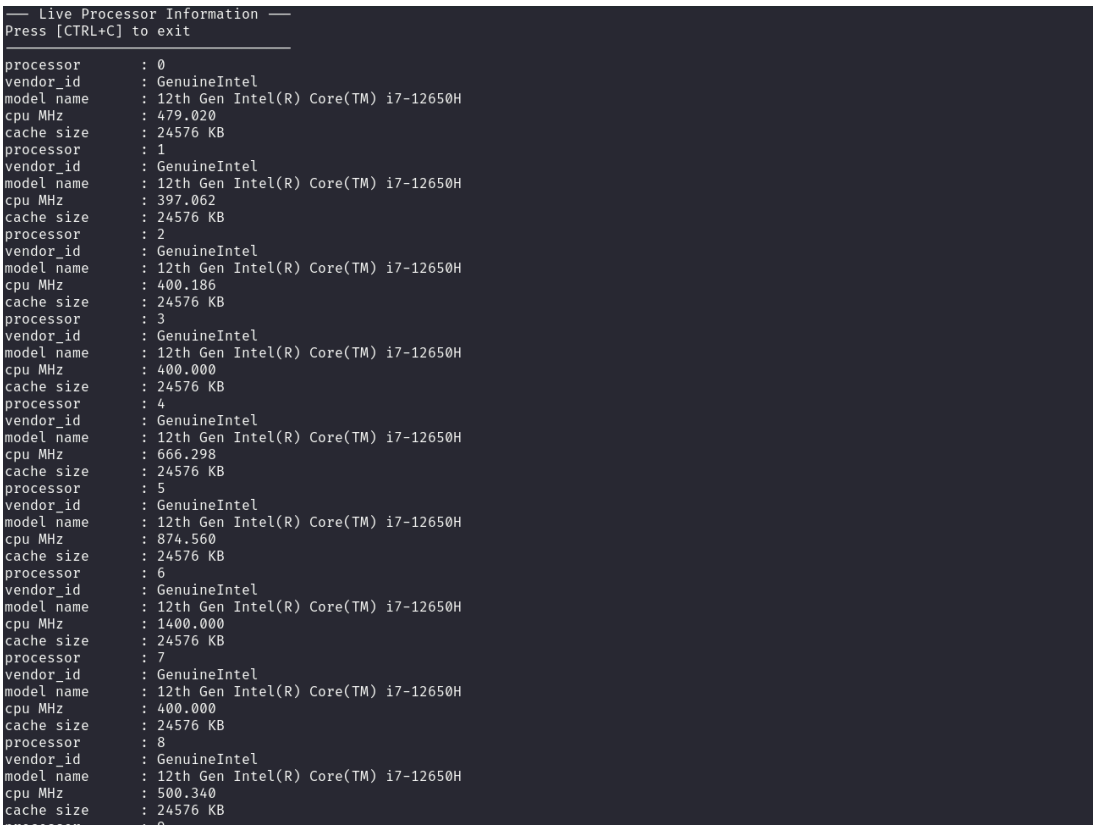
    echo "--- Live Processor Information ---"
    echo "Press [CTRL+C] to exit"
    echo "---------------------------------"

    # extract the current info for all cores
    grep -E "processor|vendor_id|model name|cpu MHz|cache size" /proc/cpuinfo

    echo "---------------------------------"

    sleep 1
done
```

Output



—

Question 8

Problem Statement:
Write a shell script multiplication_table that will generate multiplication table for a number given on the command line.

p8.sh

```
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Usage: $0 <number>"
    echo "Example: $0 7"
    exit 1
fi

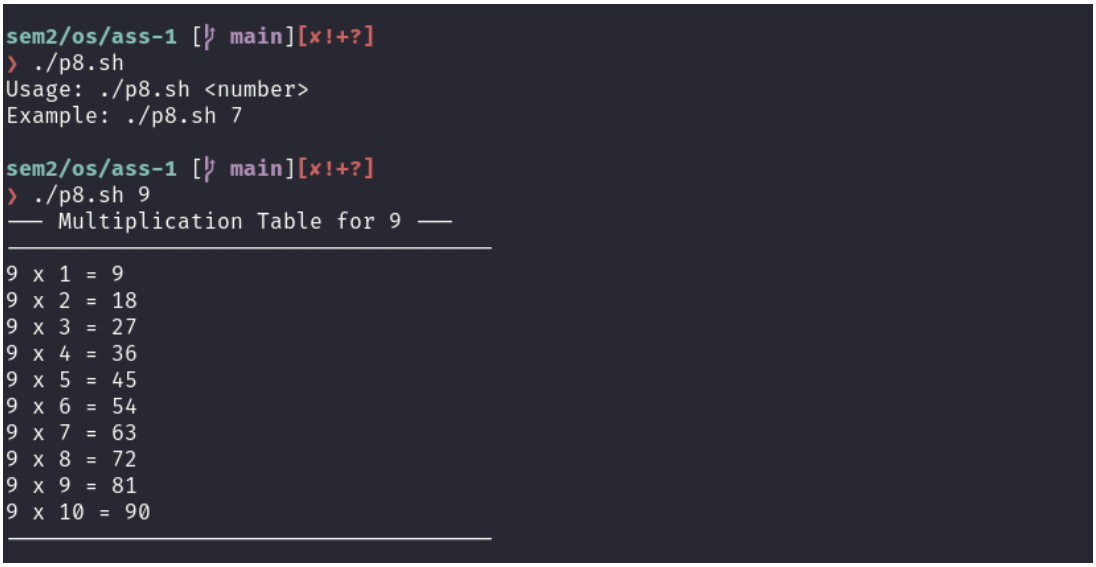
num=$1
if ! [[ "$num" =~ ^-?[0-9]+$ ]]; then
    echo "Error: '$num' is not a valid integer."
    exit 1
fi

echo "--- Multiplication Table for $num ---"
echo "-------------------------------------"

for i in {1..10}; do
    result=$((num * i))
    echo "$num x $i = $result"
done

echo "-------------------------------------"
```

Output



—

Assignment 2

Question 2

Problem Statement:
Write a C program to create a child process. The parent process must wait until the child finishes. Both the processes must print their own pid and parent pid. Additionally, the parent process should print the exit status of the child.

p2.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0) {;
        printf("[Child]  My PID: %d\n", getpid());
        printf("[Child]  My Parent's PID: %d\n", getppid());

        printf("[Child]  Exiting with status 1...\n");
        exit(1);
    }
    else {
        printf("[Parent] My PID: %d\n", getpid());
        printf("[Parent] My Parent's PID (Shell/Terminal): %d\n", getppid());
        printf("[Parent] Waiting for the child process to finish...\n");

        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            int exit_status = WEXITSTATUS(status);
            printf("[Parent] Child process finished normally.\n");
            printf("[Parent] Child's exit status was: %d\n", exit_status);
        } else {
            printf("[Parent] Child process did not terminate normally.\n");
        }
    }

    return 0;
}
```

Output

```
❯ ./o2
[Parent] My PID: 23926
[Parent] My Parent's PID (Shell/Terminal): 20161
[Parent] Waiting for the child process to finish...
[Child]  My PID: 23927
[Child]  My Parent's PID: 23926
[Child]  Exiting with status 1...
[Parent] Child process finished normally.
[Parent] Child's exit status was: 1
```

—

Question 3

Problem Statement:
Write a C program which prints prime numbers between the range 1 to 10,00,000 by creating ten child processes and subdividing the task equally among all child processes, i.e., the first child should print prime numbers in the range 1 to 1,00,000, the second child in the range 1,00,001 to 2,00,000, ... The child processes must run in parallel and the parent process must wait until all the child processes finish. [Output range reduced to 1 to 100 to fit the whole execution]

p3.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdbool.h>

bool is_prime(int n) {
    if (n <= 1) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;

    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
    }
    return true;
}

int main() {
    int num_children = 10;
    int range_size = 100000;
    pid_t pid;

    printf("[Parent] Starting 10 child processes to find prime numbers...\n\n");

    for (int i = 0; i < num_children; i++) {
        pid = fork();

        if (pid < 0) {
            perror("Fork failed");
            exit(1);
        }
        else if (pid == 0) {
            int start = (i * range_size) + 1;
            int end = (i + 1) * range_size;

            printf("[Child %d] PID: %d is checking range %d to %d\n", i + 1, getpid(), start, end);

            for (int num = start; num <= end; num++) {
                if (is_prime(num)) {
                    printf("[Child %d Found]: %d\n", i + 1, num);
                }
            }

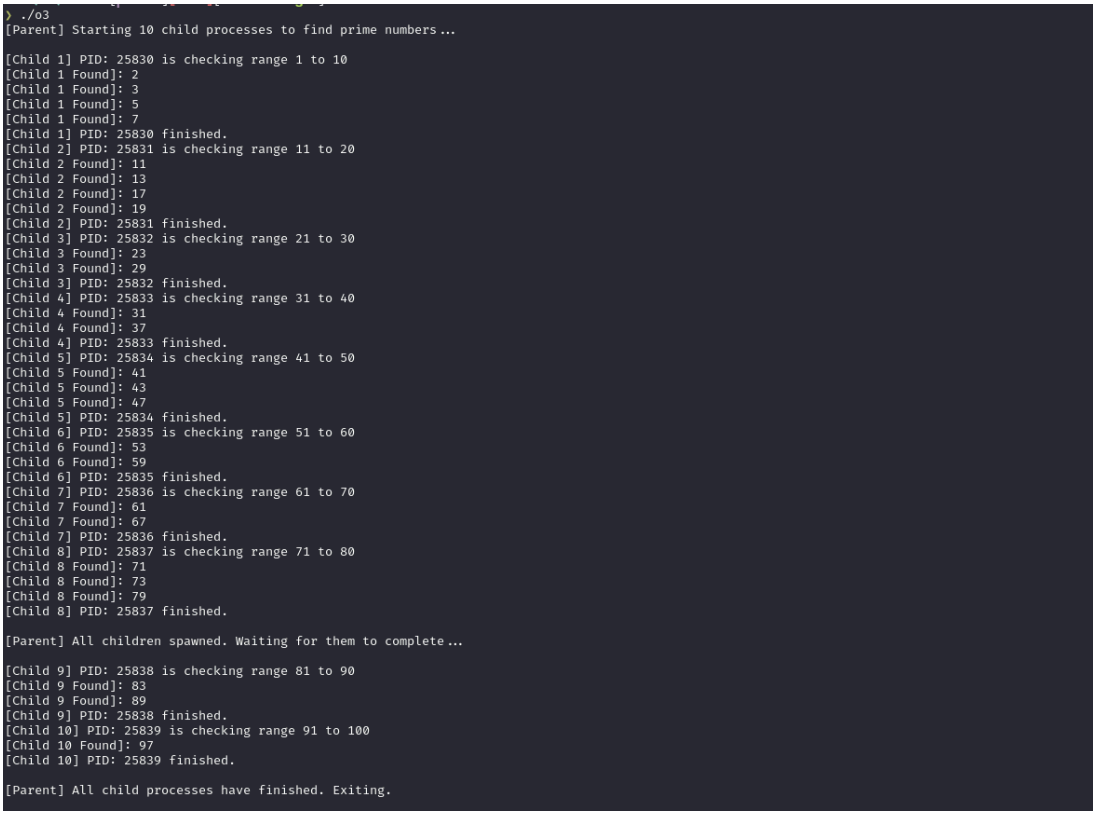
            printf("[Child %d] PID: %d finished.\n", i + 1, getpid());
            exit(0);
        }
    }

    printf("\n[Parent] All children spawned. Waiting for them to complete...\n\n");

    for (int i = 0; i < num_children; i++) {
        wait(NULL);
    }
```

```c
    printf("\n[Parent] All child processes have finished. Exiting.\n");
    return 0;
}
```

Output



—

Assignment 3

Question 5

Problem Statement:
Write a C program for simulating the Indexed File Allocation algorithm.

p5.c

```c
#include <stdio.h>

#define MAX_BLOCKS 100

int main() {
  int blocks[MAX_BLOCKS];
  int indexBlock, numFiles, numBlocks, blockNum;
  int fileBlocks[MAX_BLOCKS];

  for (int i = 0; i < MAX_BLOCKS; i++) {
    blocks[i] = 0;
  }

  printf("Enter the total number of files to allocate: ");
  scanf("%d", &numFiles);

  for (int i = 0; i < numFiles; i++) {
    printf("\n----------------------------------------");
    printf("\nProcessing File %d:\n", i + 1);
    printf("Enter the index block number: ");
    scanf("%d", &indexBlock);

    if (blocks[indexBlock] == 1) {
      printf("Error: Block %d is already allocated. Allocation failed for this file.\n",
             indexBlock);
      continue;
    }

    printf("Enter the number of data blocks needed for this file: ");
    scanf("%d", &numBlocks);

    printf("Enter the block numbers for this file:\n");
    int allocationPossible = 1;

    for (int j = 0; j < numBlocks; j++) {
      printf("Block %d: ", j + 1);
      scanf("%d", &blockNum);
      fileBlocks[j] = blockNum;

      if (blocks[blockNum] == 1 || blockNum == indexBlock) {
        allocationPossible = 0;
      }
    }

    if (allocationPossible == 0) {
      printf("\nAllocation Failed: One or more blocks are already allocated or conflict with the
index block.\n");
    } else {
      blocks[indexBlock] = 1;

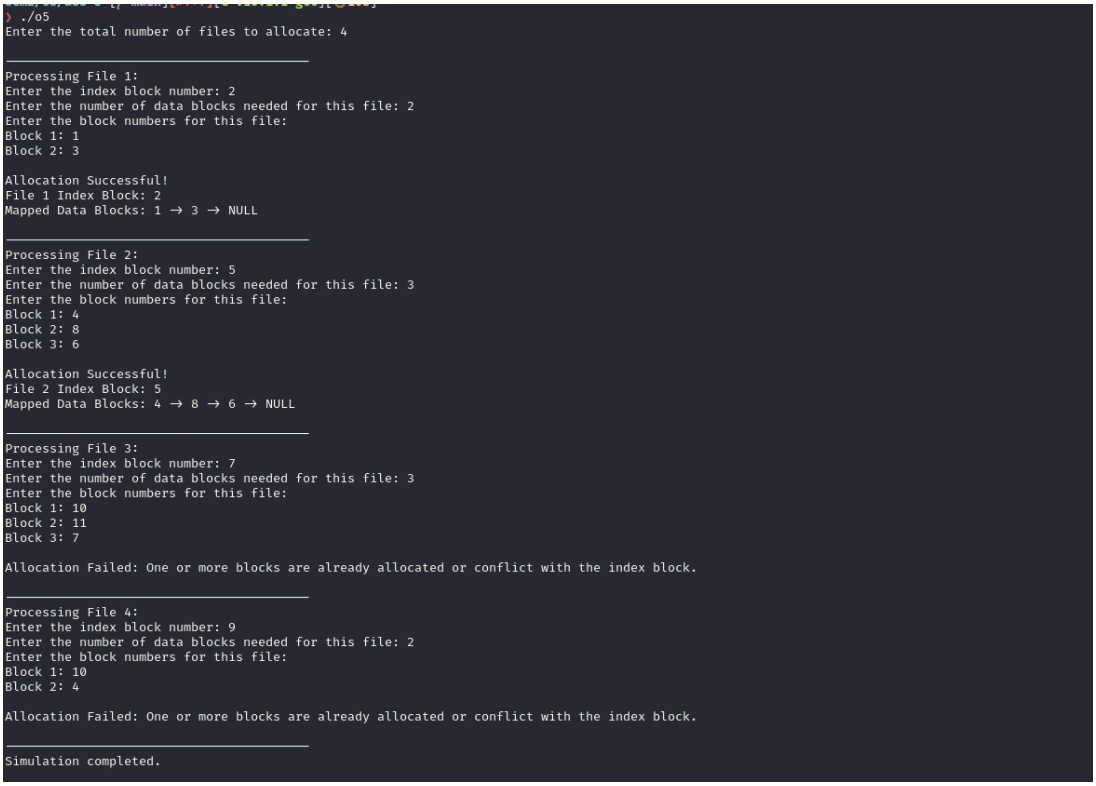
      for (int j = 0; j < numBlocks; j++) {
        blocks[fileBlocks[j]] = 1;
      }

      printf("\nAllocation Successful!\n");
      printf("File %d Index Block: %d\n", i + 1, indexBlock);
      printf("Mapped Data Blocks: ");
      for (int j = 0; j < numBlocks; j++) {
```

```
            printf("%d -> ", fileBlocks[j]);
        }
        printf("NULL\n");
      }
    }

    printf("\n----------------------------------------\n");
    printf("Simulation completed.\n");
    return 0;
}
```

Output



—

Question 7

Problem Statement:
Write a program to simulate Least Recently Used (LRU) page replacement algorithm for the following page reference string: 9, 8, 11, 7, 12, 8, 7, 6, 8, 12, 5, 4, 12, 3, 10, 11, 12, 4, 9, 4.
Consider

i. 4 frames
ii. 5 frames

Compare the results.

p7.c

```c
#include <stdio.h>

#define MAX_PAGES 50

int findLRU(int time[], int n) {
  int minimum = time[0], minIndex = 0;
  for (int i = 1; i < n; i++) {
    if (time[i] < minimum) {
      minimum = time[i];
      minIndex = i;
    }
  }
  return minIndex;
}

int simulateLRU(int pages[], int numPages, int numFrames) {
  int frames[numFrames];
  int time[numFrames];
  int pageFaults = 0;
  int counter = 0;

  for (int i = 0; i < numFrames; i++) {
    frames[i] = -1;
    time[i] = 0;
  }

  printf("\n--- Simulation with %d Frames ---\n", numFrames);
  printf("Incoming Page | Frame State\n");
  printf("--------------|----------------------\n");

  for (int i = 0; i < numPages; i++) {
    int currentPage = pages[i];
    int flag1 = 0, flag2 = 0;

    for (int j = 0; j < numFrames; j++) {
      if (frames[j] == currentPage) {
        counter++;
        time[j] = counter;
        flag1 = 1;
        flag2 = 1;
        break;
      }
    }

    if (flag1 == 0) {
      for (int j = 0; j < numFrames; j++) {
        if (frames[j] == -1) {
          counter++;
          pageFaults++;
          frames[j] = currentPage;
          time[j] = counter;
          flag2 = 1;
          break;
```

```
        }
      }
    }

    if (flag2 == 0) {
      int lruIndex = findLRU(time, numFrames);
      counter++;
      pageFaults++;
      frames[lruIndex] = currentPage;
      time[lruIndex] = counter;
    }

    printf("      %2d      | ", currentPage);
    for (int j = 0; j < numFrames; j++) {
      if (frames[j] != -1) {
        printf("%2d ", frames[j]);
      } else {
        printf("-  ");
      }
    }
    if (flag1 == 1) {
      printf(" (Hit)");
    } else {
      printf(" (Fault)");
    }
    printf("\n");
  }

  printf("Total Page Faults: %d\n", pageFaults);
  return pageFaults;
}

int main() {
  int numPages;

  printf("--- LRU Page Replacement Algorithm Simulation ---\n\n");
  printf("Enter the length of the page reference string: ");
  scanf("%d", &numPages);

  int pages[MAX_PAGES];
  printf("Enter the page reference string (separated by spaces):\n");
  for (int i = 0; i < numPages; i++) {
    scanf("%d", &pages[i]);
  }

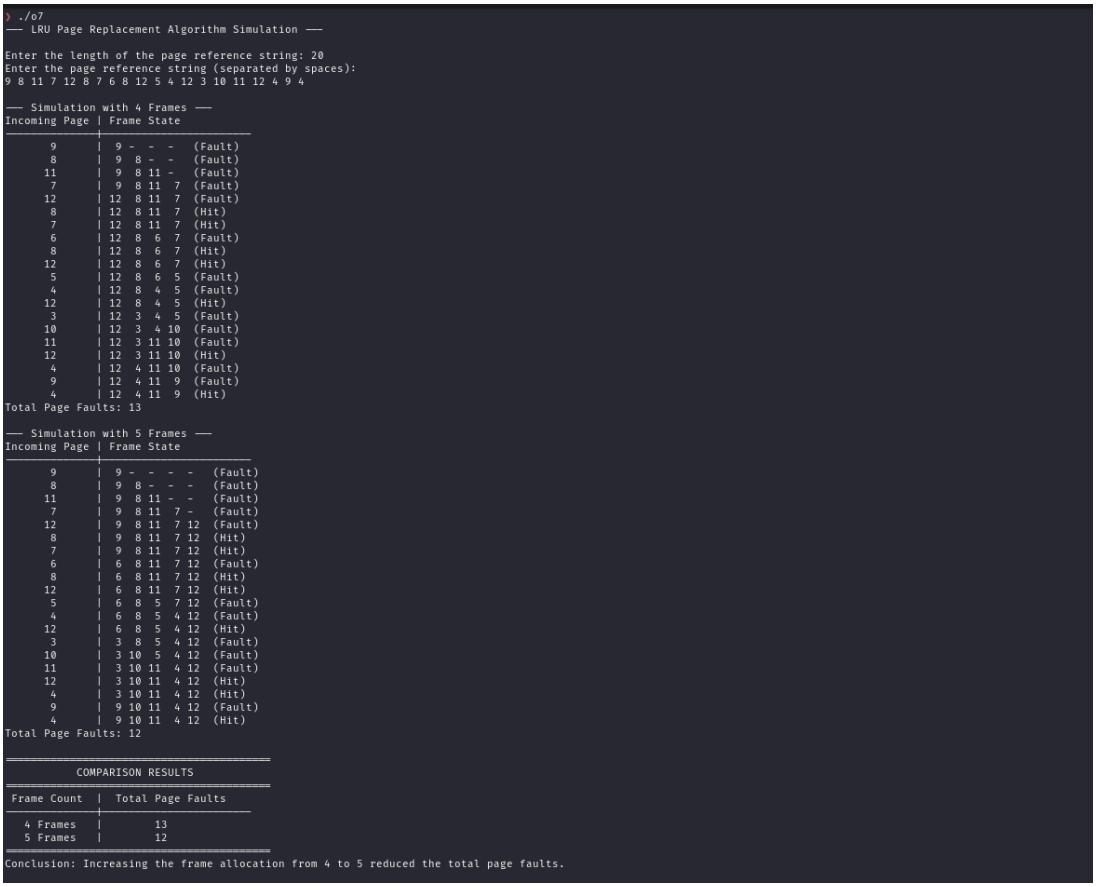
  int faultsWith4 = simulateLRU(pages, numPages, 4);
  int faultsWith5 = simulateLRU(pages, numPages, 5);

  printf("\n========================================\n");
  printf("           COMPARISON RESULTS           \n");
  printf("========================================\n");
  printf(" Frame Count  |  Total Page Faults \n");
  printf("--------------|---------------------\n");
  printf("   4 Frames   |        %d\n", faultsWith4);
  printf("   5 Frames   |        %d\n", faultsWith5);
  printf("========================================\n");

  if (faultsWith4 > faultsWith5) {
    printf("Conclusion: Increasing the frame allocation from 4 to 5 reduced the total page faults.
\n");
  } else if (faultsWith4 == faultsWith5) {
    printf("Conclusion: Both configurations resulted in the same number of page faults for this
reference string.\n");
  } else {
    printf("Conclusion: Notice that LRU strictly obeys the inclusion property, meaning page faults
will not increase when adding frames.\n");
  }
```

```
    return 0;
}
```

Output



```
❯ ./o7
── LRU Page Replacement Algorithm Simulation ──

Enter the length of the page reference string: 20
Enter the page reference string (separated by spaces):
9 8 11 7 12 8 7 6 8 12 5 4 12 3 10 11 12 4 9 4

── Simulation with 4 Frames ──
Incoming Page | Frame State
--------------+-------------------
        9     |  9  -  -  -   (Fault)
        8     |  9  8  -  -   (Fault)
       11     |  9  8 11  -   (Fault)
        7     |  9  8 11  7   (Fault)
       12     | 12  8 11  7   (Fault)
        8     | 12  8 11  7   (Hit)
        7     | 12  8 11  7   (Hit)
        6     | 12  8  6  7   (Fault)
        8     | 12  8  6  7   (Hit)
       12     | 12  8  6  7   (Hit)
        5     | 12  8  6  5   (Fault)
        4     | 12  8  4  5   (Fault)
       12     | 12  8  4  5   (Hit)
        3     | 12  3  4  5   (Fault)
       10     | 12  3  4 10   (Fault)
       11     | 12  3 11 10   (Fault)
       12     | 12  3 11 10   (Hit)
        4     | 12  4 11 10   (Fault)
        9     | 12  4 11  9   (Fault)
        4     | 12  4 11  9   (Hit)
Total Page Faults: 13

── Simulation with 5 Frames ──
Incoming Page | Frame State
--------------+-------------------
        9     |  9  -  -  -  -   (Fault)
        8     |  9  8  -  -  -   (Fault)
       11     |  9  8 11  -  -   (Fault)
        7     |  9  8 11  7  -   (Fault)
       12     |  9  8 11  7 12   (Fault)
        8     |  9  8 11  7 12   (Hit)
        7     |  9  8 11  7 12   (Hit)
        6     |  6  8 11  7 12   (Fault)
        8     |  6  8 11  7 12   (Hit)
       12     |  6  8 11  7 12   (Hit)
        5     |  6  8  5  7 12   (Fault)
        4     |  6  8  5  4 12   (Fault)
       12     |  6  8  5  4 12   (Hit)
        3     |  3  8  5  4 12   (Fault)
       10     |  3 10  5  4 12   (Fault)
       11     |  3 10 11  4 12   (Fault)
       12     |  3 10 11  4 12   (Hit)
        4     |  3 10 11  4 12   (Hit)
        9     |  9 10 11  4 12   (Fault)
        4     |  9 10 11  4 12   (Hit)
Total Page Faults: 12

=====================================
           COMPARISON RESULTS
=====================================
 Frame Count  |  Total Page Faults
--------------+--------------------
   4 Frames   |        13
   5 Frames   |        12
=====================================
Conclusion: Increasing the frame allocation from 4 to 5 reduced the total page faults.
```

—

Assignment 4

Question 2

Problem Statement:
Write a C program to simulate SCAN and CSCAN Disk scheduling algorithm.

a) Use 3 randomly generated lists of disk requests spanning from cylinders 0 (outermost) to 199
b) Test each list with each disk scheduling algorithm and compare the results
c) Generate a list of requests in which the requests are concentrated at one end of the disk
d) Generate a list of request in which the requests are concentrated at the centre of the disk

Repeat step (b) for both (c) and (d).

endGen.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define REQ_SIZE 20

int main() {
    FILE *fptr = fopen("ends_requests.txt", "w");
    if (fptr == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    srand(time(NULL));
    for (int i = 0; i < REQ_SIZE; i++) {
        int val;
        if (rand() % 2 == 0) {
            val = rand() % 41; // 50% chance of being 0 - 40
        } else {
            val = 160 + (rand() % 40); // 50% chance of being 160 - 199
        }
        fprintf(fptr, "%d ", val);
    }

    fclose(fptr);
    printf("Successfully generated ends_requests.txt\n");
    return 0;
}
```

midGen.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define REQ_SIZE 20

int main() {
    FILE *fptr = fopen("mids_requests.txt", "w");
    if (fptr == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    srand(time(NULL));
    for (int i = 0; i < REQ_SIZE; i++) {
        int val = 80 + (rand() % 41);  // 80 - 120

        fprintf(fptr, "%d ", val);
    }
```

```
    fclose(fptr);
    printf("Successfully generated mids_requests.txt\n");
    return 0;
}
```

p2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_CYLINDERS 200
#define REQ_SIZE 20
#define INITIAL_HEAD 50

void sort_array(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int simulate_scan(int requests[], int n, int head) {
    int total_movement = 0;
    int sorted[REQ_SIZE + 2];
    int size = 0;

    for(int i = 0; i < n; i++) sorted[size++] = requests[i];

    int has_higher = 0;
    for(int i = 0; i < n; i++) if(requests[i] > head) has_higher = 1;
    if(has_higher) sorted[size++] = MAX_CYLINDERS - 1;

    sort_array(sorted, size);

    int pos = 0;
    while (pos < size && sorted[pos] < head) pos++;

    int current_head = head;

    for (int i = pos; i < size; i++) {
        total_movement += abs(sorted[i] - current_head);
        current_head = sorted[i];
    }

    for (int i = pos - 1; i >= 0; i--) {
        total_movement += abs(sorted[i] - current_head);
        current_head = sorted[i];
    }
    return total_movement;
}

int simulate_cscan(int requests[], int n, int head) {
    int total_movement = 0;
    int sorted[REQ_SIZE + 2];
    int size = 0;

    for(int i = 0; i < n; i++) sorted[size++] = requests[i];

    sorted[size++] = MAX_CYLINDERS - 1;
```

```c
    sort_array(sorted, size);

    int pos = 0;
    while (pos < size && sorted[pos] < head) pos++;

    int current_head = head;

    for (int i = pos; i < size; i++) {
        total_movement += abs(sorted[i] - current_head);
        current_head = sorted[i];
    }

    if (pos > 0) {
        current_head = 0;

        for (int i = 0; i < pos; i++) {
            total_movement += abs(sorted[i] - current_head);
            current_head = sorted[i];
        }
    }
    return total_movement;
}

void evaluate_list(int req[], int n, const char* label) {
    printf("\n%s\nRequests: ", label);
    for(int i = 0; i < n; i++) printf("%d ", req[i]);
    printf("\nSCAN Total Head Movements: %d\n", simulate_scan(req, n, INITIAL_HEAD));
    printf("C-SCAN Total Head Movements: %d\n", simulate_cscan(req, n, INITIAL_HEAD));
}

int read_from_file(const char* filename, int arr[], int max_size) {
    FILE *fptr = fopen(filename, "r");
    if (fptr == NULL) {
        printf("Warning: Could not open file %s\n", filename);
        return 0;
    }
    int i = 0;
    while (i < max_size && fscanf(fptr, "%d", &arr[i]) == 1) {
        i++;
    }
    fclose(fptr);
    return i;
}

int main() {
    srand(time(NULL));
    int req[REQ_SIZE];

    printf("Random Distribution\n");
    for (int l = 1; l <= 3; l++) {
        for (int i = 0; i < REQ_SIZE; i++) {
            req[i] = rand() % MAX_CYLINDERS;
        }
        char label[20];
        sprintf(label, "Random List %d", l);
        evaluate_list(req, REQ_SIZE, label);
    }

    printf("\nFile Distributions\n");

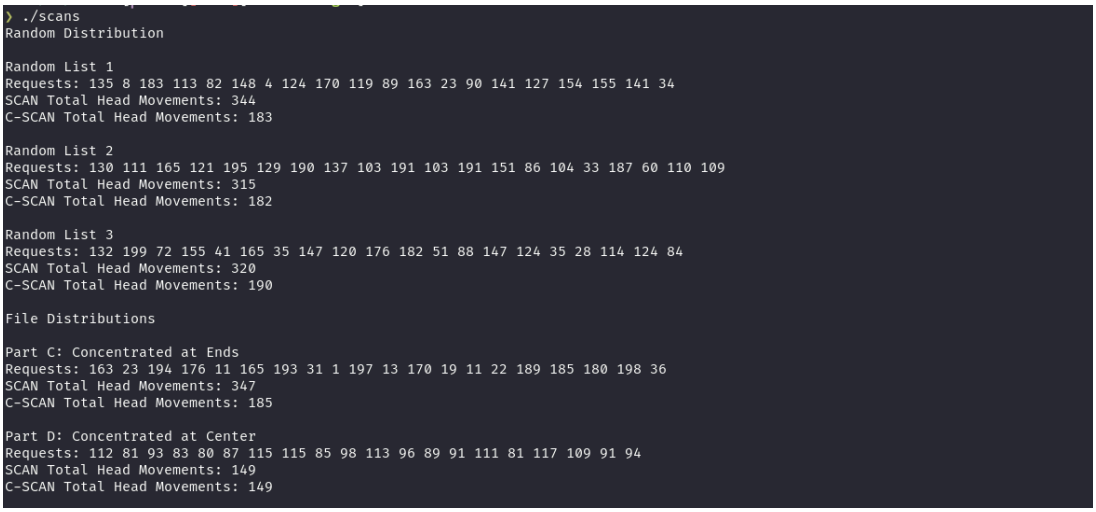
    int ends_req[REQ_SIZE];
    if (read_from_file("ends_requests.txt", ends_req, REQ_SIZE) > 0) {
        evaluate_list(ends_req, REQ_SIZE, "Part C: Concentrated at Ends");
    }

    int mids_req[REQ_SIZE];
    if (read_from_file("mids_requests.txt", mids_req, REQ_SIZE) > 0) {
        evaluate_list(mids_req, REQ_SIZE, "Part D: Concentrated at Center");
```

```
    }

    return 0;
}
```

Output



—